BLACK VAULT

Sistema de gestión documental

Manual técnico

Arquitectura, modelo de datos, API, instalación y diagramas UML

Contenido

1. Introducción 3
1.1 Alcance 3
1.2 Tecnologías 3
2. Arquitectura 3
2.1 Servidor (black-vault-express-server) 3
2.2 Cliente (black-vault-angular-client) 4
3. Modelo de datos 4
3.1 Colecciones 5
3.2 Decisiones de diseño 5
3.3 Índices 5
3.4 Validación en la base de datos 6
4. Seguridad y permisos 6
5. API REST 7
6. Reglas de negocio principales 7
7. Historial y reportes 8
8. Diagramas UML 8
8.1 Casos de uso 8
9. Instalación y puesta en marcha 9
9.1 Requisitos 9
9.2 Servidor 9
9.3 Cliente 9
9.4 Variables de entorno (.env del servidor) 9
9.5 Scripts 9
10. Datos de demostración y pruebas 10

1. Introducción

Black Vault es un sistema de gestión documental para almacenar, organizar, consultar y compartir archivos de texto (`.txt` y Markdown `.md`) entre los usuarios de una organización. Este manual describe cómo está construido, cómo se instala y cómo se verifica. El uso de la aplicación se explica en el **Manual de usuario**.

1.1 Alcance

- Usuarios con dos roles (administrador y usuario), inicio de sesión con JWT y cuentas que se desactivan sin perder su información.
- Carpetas anidadas, archivos con categoría, etiquetas y descripción, vista previa, edición en línea, descarga y búsqueda.
- Compartición de archivos y carpetas con tres niveles de permiso (lectura, comentario, edición) y herencia a todo el contenido de una carpeta.
- Papelera con restauración y eliminación definitiva, historial de actividad, 12 reportes y una verificación de integridad de los datos.

1.2 Tecnologías

Capa	Tecnología	Uso
Cliente	Angular 21 + Angular Material 21	Aplicación de una sola página (SPA), standalone, zoneless y con signals
Servidor	Node.js 24 + Express 5	API REST en `/api`, validación con zod, subidas con Multer
Base de datos	MongoDB 9.0 (driver oficial 7.7)	Base `black_vault`; el contenido de los archivos va en GridFS
Seguridad	jsonwebtoken, bcryptjs, helmet, express-rate-limit	Sesiones JWT, contraseñas con hash, cabeceras seguras, límite de intentos
Pruebas	node:test + supertest; Vitest (Angular)	Pruebas automáticas del servidor y del cliente

2. Arquitectura

El sistema sigue una arquitectura cliente-servidor de tres capas. El navegador descarga la aplicación Angular, que se comunica con la API por HTTP usando JSON y un token JWT en la cabecera `Authorization`. Solo la API se conecta a MongoDB: el cliente nunca accede a la base de datos.

2.1 Servidor (black-vault-express-server)

Está organizado por módulos. Cada módulo tiene sus rutas, su lógica (servicio) y sus esquemas de validación.

Carpeta	Contenido
`src/servidor.js`, `src/app.js`	Arranque, middlewares (helmet, CORS, JSON, registro), rutas, 404 y manejo de errores
`src/config/`	Variables de entorno validadas al arrancar

Carpeta	Contenido
src/bd/	Conexión, bucket GridFS, validadores e índices, operaciones atómicas
src/middlewares/	Autenticación, rol requerido, validación (zod), subida de archivos, errores
src/modulos/	autenticacion, usuarios, carpetas, archivos, comparticiones, papelera, actividades, almacenamiento, reportes, mantenimiento, salud
src/servicios/	Permisos, historial, cuota, GridFS, contraseñas, tokens, integridad
scripts/	Carga inicial (semilla.js y sus datos) y verificación de integridad
pruebas/	Pruebas automáticas de la API (base aparte: black_vault_pruebas)

2.2 Cliente (black-vault-angular-client)

Carpeta	Contenido
nucleo/	Modelos, servicios de acceso a la API, interceptores (token y errores) y guardias de ruta
compartido/	Componentes reutilizables: diálogos, ruta de carpetas, gráficas, lista de actividad, exportación CSV
diseno/	Estructura de la pantalla: barra superior, menú lateral e indicador de espacio
paginas/	inicio-sesion, mi-unidad, visor, compartidos, busqueda, papelera, actividad, perfil, administracion

Las páginas se cargan bajo demanda (carga diferida) y todo el estado se maneja con signals. Las descargas se piden con HttpClient porque el token va en la cabecera; el Markdown se muestra con marked y la sanitización de Angular.

3. Modelo de datos

La base black_vault tiene cinco colecciones y el bucket GridFS contenidos. Se combinan referencias (para relaciones que crecen sin límite o son de muchos a muchos) y documentos embebidos (para datos que siempre se leen junto con su dueño).

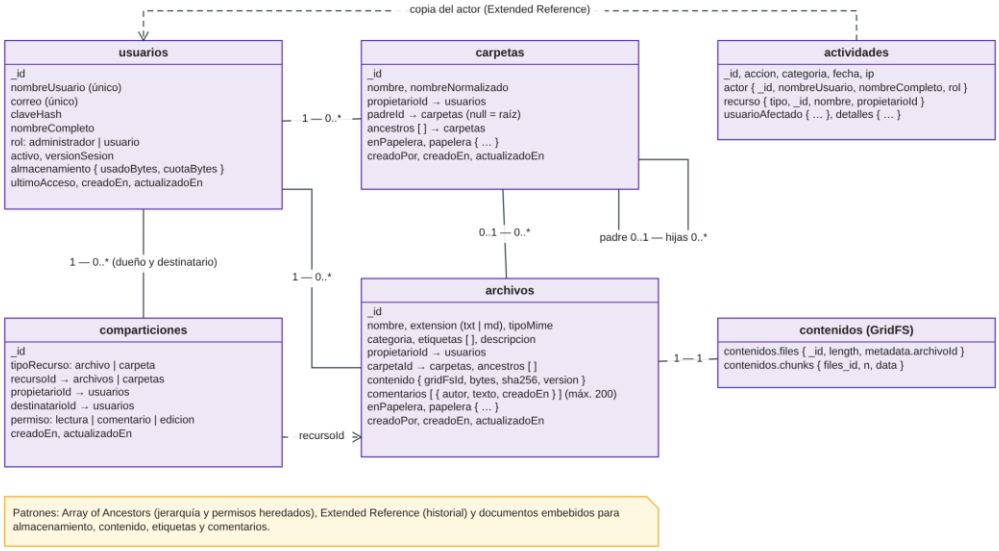


Figura 2. Modelo documental: colecciones y relaciones

3.1 Colecciones

Colección	Guarda	Embebido	Referencias
usuarios	Cuentas, rol, estado y espacio	`almacenamiento` { usadoBytes, cuotaBytes }	—
carpetas	Jerarquía de carpetas	`papelera` (estado)	`propietarioId`, `padreId`, `ancestros[]`
archivos	Metadatos de cada archivo	`contenido`, `etiquetas[]`, `comentarios[]` (máx. 200), `papelera`	`propietarioId`, `carpetaId`, `ancestros[]`, `contenido.gridFsId`
comparticiones	Permisos otorgados	—	`recursoId`, `propietarioId`, `destinatarioId`
actividades	Historial de acciones	Copia de `actor`, `recurso` y `usuarioAfectado`	`_id` dentro de cada copia
contenidos.files / .chunks	Bytes de los archivos (GridFS)	`metadata` { archivoId, propietarioId, tipoMime, sha256 }	`metadata.archivoId`

3.2 Decisiones de diseño

- **Array of Ancestors** en carpetas y archivos: `ancestros` guarda todas las carpetas que contienen al elemento. Con una consulta indexada se obtiene un subárbol completo (papelera en cascada, mover, eliminar), se arma la ruta, se detectan ciclos y se calculan los permisos heredados.
- **GridFS** para el contenido: separa los bytes de los metadatos (los listados no cargan contenido), no tiene el límite de 16 MB de un documento y permite leer en streaming. El `sha256` comprueba que el archivo se recupera sin cambios.
- **Comentarios embebidos con tope de 200**: solo tienen sentido dentro de su archivo y desaparecen con él; el tope evita que el documento crezca sin límite.
- **Comparticiones en colección propia**: es una relación de muchos a muchos que se consulta desde los dos lados ("compartidos conmigo" y "por mí") y alimenta cuatro reportes.
- **Extended Reference en el historial**: cada actividad copia el nombre del actor y del recurso en ese momento, así el historial se conserva aunque el archivo se renombre o se elimine y aunque el usuario se desactive. Los reportes no necesitan `$lookup`.
- **Espacio embebido en el usuario**: un `$inc` condicional sobre un solo documento es atómico, así que el espacio usado nunca supera la cuota.

3.3 Índices

Se crean 25 índices al arrancar el servidor y en la carga inicial. Los principales:

Colección	Índice	Para qué sirve
usuarios	`nombreUsuario` y `correo` (únicos)	Inicio de sesión y unicidad
carpetas	{ padreId, propietarioId, nombreNormalizado } único parcial (activas)	Sin nombres repetidos entre carpetas hermanas; listado ordenado
carpetas, archivos	`ancestros` (multikey)	Subárboles y búsqueda dentro de carpetas compartidas

Colección	Índice	Para qué sirve
archivos	{ carpetaId, propietarioId, nombreNormalizado } único parcial	Sin nombres repetidos en una carpeta
archivos	Índice de texto: nombre (10), etiquetas (5), descripción (1)	Búsqueda por palabras con relevancia, en español
archivos	`etiquetas`, `creadoEn`, { propietarioId, enPapelera, creadoEn }	Filtros, mis archivos y reporte por fechas
carpetas, archivos	{ propietarioId, papelera.esRaiz, papelera.enviadoEn } y `papelera.raizId` (parciales)	Vista de papelera y restaurar en grupo
comparticiones	{ recursoId, destinatarioId } único; destinatario y propietario por fecha	Una compartición por persona; permiso efectivo; compartidos conmigo y por mí
actividades	actor, recurso, propietario del recurso, afectado, acción y fecha	Historial por usuario y por recurso, filtros y reportes
contenidos.files	`metadata.archivoId`	Verificar que no haya contenido huérfano

3.4 Validación en la base de datos

Cada colección tiene un validador `$jsonSchema` (campos requeridos, tipos, valores permitidos) y reglas `$expr` que cruzan campos: el espacio usado no supera la cuota, una carpeta no puede ser su propio ancestro, el último ancestro es siempre la carpeta padre y nadie comparte consigo mismo. Así la integridad se garantiza también si alguien escribe en la base sin pasar por la API.

4. Seguridad y permisos

- Las contraseñas se guardan con **bcrypt** (costo 10) y nunca salen en las respuestas. Mínimo 8 caracteres con letras y números.
- Al iniciar sesión, la API entrega un **JWT** (vigencia de 8 horas) que incluye `versionSesion`. Cambiar o restablecer la contraseña, cambiar el rol o desactivar la cuenta aumenta esa versión e invalida los tokens anteriores.
- El inicio de sesión admite **10 intentos fallidos** cada 15 minutos por dirección; además se usan `helmet` y CORS limitado al origen del cliente.
- El servidor comprueba el permiso en **cada** operación. Sin acceso responde **404** (no revela que el recurso existe); con un permiso o rol insuficiente responde **403**.

El **permiso efectivo** es el mayor entre la compartición directa del recurso y las de cualquiera de sus carpetas ancestro. El propietario siempre tiene control total.

Acción	Propietario	Edición	Comentario	Lectura
Ver, navegar, vista previa, descargar y ver comentarios	✓	✓	✓	✓
Agregar comentarios	✓	✓	✓	—
Editar contenido, renombrar y clasificar; subir o crear dentro de una carpeta compartida	✓	✓	—	—
Mover, enviar a la papelera, restaurar y eliminar definitivamente	✓	—	—	—

Acción	Propietario	Edición	Comentario	Lectura
Compartir, cambiar y retirar permisos; ver quién tiene acceso	✓	—	—	—

5. API REST

Todas las rutas empiezan con `/api` y, salvo iniciar sesión y salud, requieren el token. Los errores tienen siempre la forma `{ "error": { "codigo", "mensaje", "detalles" } }` con los códigos 400 VALIDACION, 401 NO_AUTENTICADO, 403 PROHIBIDO, 404 NO_ENCONTRADO, 409 CONFLICTO o CUOTA_EXCEDIDA, 413 ARCHIVO_DEMASIADO_GRANDE y 415 TIPO_NO_PERMITIDO. Los listados se paginan con `?pagina=&limite=` (máximo 100).

Módulo	Rutas principales	Quién
Salud	`GET /salud`	Público
Autenticación	`POST /autenticacion/iniciar-sesion`, `POST /autenticacion/cerrar-sesion`, `GET /autenticacion/perfil`, `PATCH /autenticacion/clave`	Público / sesión
Usuarios	`GET, POST /usuarios`, `GET, PATCH /usuarios/:id`, `PATCH /usuarios/:id/estado`, `PATCH /usuarios/:id/clave`; `GET /usuarios/directorio`	Administrador; el directorio, cualquier usuario
Carpetas	`GET /carpetas/raiz/contenido`, `GET /carpetas/arbol`, `POST /carpetas`, `GET /carpetas/:id`, `GET /carpetas/:id/contenido`, `PATCH /carpetas/:id`, `PATCH /carpetas/:id/mover`, `DELETE /carpetas/:id`	Según el permiso
Archivos	`GET /archivos` (búsqueda y filtros), `POST /archivos` (multipart), `GET /archivos/:id`, `/contenido`, `/descarga`, `PUT /archivos/:id/contenido`, `PATCH /archivos/:id`, `/mover`, `DELETE /archivos/:id`, comentarios	Según el permiso
Comparticiones	`GET /comparticiones/recurso/:tipo/:id`, `POST`, `PATCH`, `DELETE /comparticiones/:id`, `GET /comparticiones/conmigo`, `/realizadas`	Propietario; listados, cualquiera
Papelera	`GET /papelera`, `POST /papelera/:tipo/:id/restaurar`, `DELETE /papelera/:tipo/:id`, `DELETE /papelera`	Propietario
Actividad	`GET /actividades`, `GET /actividades/recurso/:tipo/:id`	Según el rol
Otros	`GET /almacenamiento`, `GET /reportes/...` (13 rutas), `GET /mantenimiento/integridad`	Usuario; administrador

6. Reglas de negocio principales

- **Subir**: solo `.txt` y `.md`, hasta 5 MB, en UTF-8 (con o sin BOM; un archivo en Latin-1 se acepta, pero no se puede editar en línea). El espacio se reserva antes de guardar y se devuelve si algo falla.
- **Nombres**: no se repiten dentro de una carpeta (sin distinguir mayúsculas ni tildes). Si al subir ya existe, se agrega un número: "Informe (1).md".
- **Editar**: control de versión optimista; si otra persona guardó antes, la API responde 409 y el cliente avisa en lugar de sobrescribir. Se conservan el BOM y los saltos de línea originales.

- **Mover una carpeta**: no se permite moverla dentro de sí misma; los ancestros de todo su contenido se actualizan en la misma operación.
- **Papelera**: enviar una carpeta envía todo su contenido; lo que está en la papelera no aparece en listados, búsquedas ni para las personas con quienes se compartió, pero sigue ocupando espacio. Restaurar devuelve el elemento a su lugar (o a la raíz si su carpeta ya no existe) y lo renombra si hay conflicto. Eliminar definitivamente borra el contenido de GridFS, las compartciones y libera el espacio.
- **Operaciones atómicas**: si MongoDB es un replica set, las operaciones compuestas usan transacciones; en una instalación normal se ejecutan en orden con compensación.
- **Usuarios**: no se eliminan, se desactivan; un administrador no puede desactivarse a sí mismo ni dejar el sistema sin administradores activos, y la cuota no puede quedar por debajo de lo ya usado.

7. Historial y reportes

Cada operación registra una actividad con su acción, categoría (autenticación, usuarios, carpetas, archivos, papelera, comparticiones), fecha, actor, recurso, detalles e IP. Un usuario ve lo que hizo, lo que otros hicieron con sus recursos y lo que hicieron con él; el administrador ve todo, con la IP.

Los reportes se calculan con aggregation pipelines dentro de MongoDB y solo los consulta el administrador. Las fechas se interpretan en hora de Guatemala.

La **verificación de integridad** (Administración → Integridad o `npm run integridad`) revisa el espacio usado, la papelera, la jerarquía de carpetas, el contenido en GridFS y las comparticiones.

8. Diagramas UML

Los diagramas están en `docs/uml/` en formato draw.io (`.drawio`), uno por diagrama y uno con todas las páginas (`black-vault-uml.drawio`). Se abren en diagrams.net o en la extensión Draw.io de VS Code.

8.1 Casos de uso



Figura 3. Casos de uso

9. Instalación y puesta en marcha

9.1 Requisitos

- Windows 10/11 (u otro sistema con las mismas versiones), Node.js 24.12 o superior.
- MongoDB 9.0 ejecutándose en `127.0.0.1:27017`. MongoDB Compass es opcional, para explorar los datos.

9.2 Servidor

```
cd black-vault-express-server
copy .env.example .env
npm install
npm run semilla
npm run dev
```

En `.env`, cambia `JWT_SECRETO` por una cadena aleatoria de 32 caracteres o más. `npm run semilla` borra y vuelve a crear la base con los datos de demostración (pide escribir SI). La API queda en http://localhost:8000/api/salud.

9.3 Cliente

```
cd black-vault-angular-client
npm install
npm start
```

La aplicación queda en http://localhost:4200.

9.4 Variables de entorno (.env del servidor)

Variable	Valor por defecto	Descripción
`PUERTO`	8000	Puerto de la API
`URI_MONGODB`	mongodb://127.0.0.1:27017/black_vault	Conexión a MongoDB (admite usuario, clave y replicaSet)
`JWT_SECRETO`	— (obligatoria)	Clave para firmar las sesiones, de 32 caracteres o más
`JWT_EXPIRACION`	8h	Vigencia de la sesión
`ORIGEN_CLIENTE`	http://localhost:4200	Orígenes permitidos por CORS
`TAMANO_MAXIMO_MB`	5	Tamaño máximo por archivo
`CUOTA_POR_DEFECTO_MB`	100	Espacio de cada usuario nuevo
`ZONA_HORARIA`	America/Guatemala	Zona para fechas y reportes

9.5 Scripts

Comando	Qué hace
`npm run dev` / `npm start`	Arranca la API (con recarga automática en `dev`). Si la base no existe, crea colecciones, validadores e índices

Comando	Qué hace
`npm run semilla`	Reconstruye la base con los datos de demostración y verifica la integridad (`-- --forzar` no pregunta)
`npm run integridad`	Revisa la consistencia de la base (`-- --reparar` corrige lo que se puede corregir)
`npm test`	Pruebas del servidor sobre la base `black_vault_pruebas`
`npx ng test --watch=false`	Pruebas del cliente

10. Datos de demostración y pruebas

La carga inicial crea **12 usuarios** (1 administrador y 1 cuenta desactivada), **38 carpetas** de hasta tres niveles, **57 archivos** en GridFS (6 en la papelera), **34 comparticiones** con los tres permisos y herencia, **20 comentarios** y **717 actividades** de mayo a octubre de 2026, coherentes con los datos. Las contraseñas son `Admin2026` para `admin` y `Usuario2026` para los demás.

Pruebas automáticas: **128** en el servidor (autenticación, usuarios, carpetas, archivos, la matriz de permisos completa, papelera, actividad, reportes e integridad) y **28** en el cliente. Además se verificó en el navegador el recorrido completo de cada fase.